Programmable Matrix Mathematics for Industrial Systems with paceval

Complete dense matrix operations, covariance models, and general 512- and 1024-dimensional Cholesky decomposition and inversion

paceval Mathematical Research Demonstrator

13 November 2026



Figure 1: Industrial matrix execution graph concept

Research proposal and independently verified computational demonstrators. The illustration is conceptual, not a photograph of a deployment.

Executive summary

The industrial research opportunity is to use **reusable mathematical execution graphs** for configurable matrix-related computations, particularly where formulas, calibration rules, transformations and numerical decision functions change frequently. The local paceval C/C++ API is evaluated as a mathematical component, not as a proposed replacement for state-of-the-art linear algebra libraries.

This study combines the previously completed large-matrix experiments with a **new, complete, general-purpose Cholesky and inverse algorithm** for full, dense symmetric positive-definite (SPD) matrices. The new implementation actually computes all entries of the Cholesky factor and inverse for **512 × 512 and 1024 × 1024** matrices using persistent paceval graphs and `dGetComputationResultExt` batches. It does not rely on the diagonal-plus-low-rank Woodbury identity used in an earlier, narrower study.

The maximum complete-matrix residuals $\|AA^{-1}-I\|_{\max}$ are approximately 4.88×10^{-15} (512) and 7.06×10^{-15} (1024). An independent NumPy/LAPACK inversion confirms the complete outputs with maximum entrywise deviations of 2.22×10^{-15} and 3.33×10^{-15}. The matrices are real, dense, full-rank SPD examples with

measured two-norm condition numbers of about 3.28 and 2.54. These are well-conditioned test instances, not a proof of numerical reliability for every SPD matrix.

Performance remains the principal limitation. The three paceval compute stages require approximately **15.31 seconds** for 512 and **172.41 seconds** for 1024, excluding generation, graph setup and independent verification. Highly optimized specialized solvers are much faster on the same synthetic inputs. The useful scientific outcome is therefore the demonstrated breadth of paceval's programmable matrix capabilities and a falsifiable basis for studying mathematical customization, model reuse and integration costs in industrial workflows.

1. Industrial research question

Robotics, machine condition monitoring, multi-sensor fusion, numerical calibration and digital twins use matrix primitives inside much larger decision pipelines. The primary industrial matrix operations are well served by mature implementations; adding a general expression engine is only useful when an independently measurable capability is introduced. Examples include frequently changing, application-specific post-transformations, nonlinear quality scores, analytic calibration maps or versioned constraints that must be applied to many matrix-derived values.

The initial examples used one paceval computation object per output entry. This is conceptually clear but scales poorly with matrix dimensions. The larger experiments instead reuse one or a small number of mathematical graphs across many sets of variables, minimizing graph-creation overhead while exercising the actual paceval batch API. General SPD Cholesky and inversion also require strict dependency ordering between computational steps, handled by native C++ orchestration.

Research hypothesis: reusable mathematical graphs are a suitable, reproducible execution layer for explicitly defined operations surrounding matrix analysis, even where a tuned linear-algebra routine should remain the preferred implementation of the matrix primitive itself.

2. Previously executed large-matrix experiments

2.1 Dense matrix multiplication

For two general dense matrices, the numerical kernel is

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

The existing C++ demonstrator constructs one dot-product paceval graph with $2n$ scalar variables, and evaluates it repeatedly for every requested row/column pair. The graph is reused, and input values are passed through batches. A separate blocked C++ matrix multiplication provides the reference.

Dimension	Entries evaluated with paceval	paceval batch evaluation	Optimized native C++	Maximum difference to native
512 × 512	262,144	13.98 s	0.0376 s	0 in this execution path
1024 × 1024	1,048,576	124.05 s	0.3189 s	0 in this execution path

The zero reported differences are a consequence of the particular floating-point evaluation order, not evidence of exact arithmetic. Additional `long double` references quantify rounding effects. These measurements demonstrate correctness and graph reuse, not a performance advantage.

2.2 Full covariance transformation with structured rotations

The earlier test evaluated every entry of

$$\Sigma' = R\Sigma R^{\mathsf{T}},$$

where R is a **block-diagonal** orthogonal matrix formed from independent 2×2 rotations. A reusable eight-variable paceval graph evaluates each transformed entry from the corresponding 2×2 neighborhood. The full 1024×1024 result required about **1.67 s** with paceval, compared with **0.00169 s** for an equivalent specialized C++ transformation. Trace and symmetry were checked; independent NumPy results were compared. This is **not** a benchmark for an arbitrary dense rotation matrix.

2.3 Structured SPD inversion

The earlier inverse experiment used the exact structure

$$A = D + UU^{\mathsf{T}}, \qquad D_{ii} > 0, \quad U \in \mathbb{R}^{n\times 3},$$

which permits the Woodbury formula

$$A^{-1} = D^{-1} - D^{-1}U\left(I_3 + U^{\mathsf{T}}D^{-1}U\right)^{-1}U^{\mathsf{T}}D^{-1}.$$

Every resulting entry was computed with paceval. Complete 512 and 1024 results were verified with residuals near 2.22×10^{-16}; measured paceval times were about 0.55 and 2.23 seconds. This result remains useful for **low-rank updates** but must not be called generic dense inversion. The new experiment below specifically closes this gap.

3. New experiment: fully dense SPD Cholesky decomposition

3.1 Scope and test matrices

The algorithm accepts a generic real, symmetric positive-definite matrix. A valid numerical experiment must start with matrices for which positive definiteness and an independent reference are available. We construct each test matrix from a **full lower-triangular generating factor**, with nonzero dense off-diagonal elements and a positive diagonal, and form

$$A = L_0 L_0^{\mathsf{T}}.$$

The values of L_0 are generated reproducibly from trigonometric functions of both row and column indices. Unlike $D + UU^{\mathsf{T}}$ with three columns in U, this construction uses **n independent columns** and does not exploit a fixed small-rank correction. Once A has been assembled, paceval is supplied the **dense matrix entries**, not the precomputed factor L_0. Thus the production routine must actually recover the Cholesky factor.

These two data sets have full dimension and are well-conditioned. Because every finite test suite covers particular cases, the study does not claim to verify every arbitrary SPD matrix, indefinite inputs, near-singular matrices or substantially different scaling regimes.

3.2 Reusable paceval graph formulation

The factorization is

$$A = LL^{\mathsf{T}}, \qquad L_{ij} = 0 \quad \text{for } j > i, \quad L_{ii} > 0.$$

A right-looking unblocked Cholesky method updates the lower-triangular trailing matrix. At pivot k, its scalar mathematical operations are

$$L_{kk} = \sqrt{S_{kk}}, \qquad L_{ik} = \frac{S_{ik}}{L_{kk}}, \qquad S_{ij} \leftarrow S_{ij} - L_{ik}L_{jk} \quad (i \geq j > k).$$

The paceval implementation prepares persistent graphs `sqrt(a)`, `a/b` and `a-b*c`. Each scalar operation is evaluated by the **real** `pacevalLibrary_dGetComputationResultExt` API, with batched input vectors and checked per-value error codes. Native C++ controls the pivot order, input assembly, matrix storage and numerical validation; it does not silently replace these arithmetic kernels with native formulas.

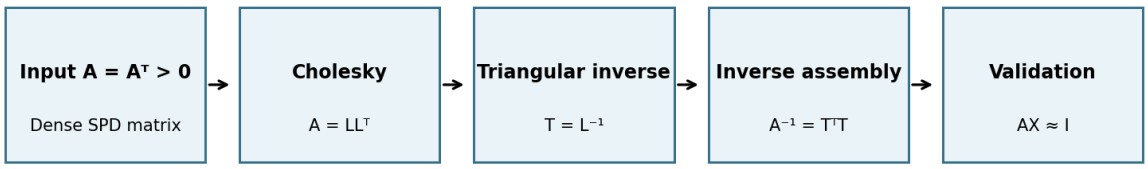


Figure 2: General dense SPD factorization and inverse workflow

The method uses only **five distinct formula graphs** across the entire factorization and inversion pipeline. Its main cost is the large number of evaluations, not creation of enormous numbers of computation objects. Pivot dependencies cannot be parallelized away by evaluating future columns before their Schur complements are available.

4. New experiment: general SPD inverse through triangular solves

The Cholesky factor is sufficient to compute the full inverse without specialized low-rank identities:

$$T = L^{-1}, \qquad A^{-1} = L^{-\mathsf{T}} L^{-1} = T^{\mathsf{T}} T.$$

The implementation uses forward substitution to construct T in place. For each row k, after prior row contributions have been applied, it divides the row by its diagonal:

$$T_{kj} \leftarrow T_{kj} / L_{kk} \quad (j \le k).$$

It then updates all subsequent rows:

$$T_{ij} \leftarrow T_{ij} - L_{ik} T_{kj} \quad (i > k,\ j \le k).$$

These divisions and subtract-multiply expressions are executed with the same reusable paceval graphs. Following construction of T, the inverse is assembled from the Gram product of its columns:

$$(A^{-1})_{ij} = \sum_{k=\max(i,j)}^{n-1} T_{ki} T_{kj}.$$

A persistent graph `a+b*c` evaluates each individual accumulation. Every unique triangular entry is computed with paceval and mirrored by C++ to produce the complete symmetric $n \times n$ inverse. The additional graph `ln(a)` supplies the log-determinant:

$$\log\det(A) = 2 \sum_{i=1}^{n} \log L_{ii}.$$

This is an **actual inversion** of dense SPD matrices, not inversion of many independent 3×3 matrices, not a reused structured inverse and not a demonstration limited to determinant or selected output elements.

5. Measured results for general dense SPD matrices

5.1 Complete execution data

Measured quantity	512 × 512	1024 × 1024
Cholesky, paceval	**5.635 s**	**58.022 s**
Triangular inverse, paceval	**5.105 s**	**58.894 s**
Full inverse assembly, paceval	**4.565 s**	**55.499 s**
Three paceval stages combined	**15.306 s**	**172.414 s**
Five paceval graph creation times combined	0.00016 s	0.00051 s
Cholesky, independent NumPy/LAPACK	0.0202 s	0.0632 s
Dense inverse, independent NumPy/LAPACK	0.0255 s	0.1130 s
Validated inverse entries	262,144	1,048,576
Maximum $\lvert AA^{-1} - I\rvert$, complete native residual	4.88×10^{-15}	7.06×10^{-15}
Maximum inverse-entry difference to independent NumPy	2.22×10^{-15}	3.33×10^{-15}

Times were measured in the available x86-64 Linux research environment with an AMD EPYC 9V74 virtual CPU. Runs are individual observations, not robust statistics over repeated hardware-controlled trials. The two paceval runs used batch capacities of 4096 and 1024 respectively; **the scaling ratios therefore cannot be treated as controlled batch-size scaling measurements**. The NumPy measurements use optimized native numerical libraries and are intended as demanding real-world reference points, not as equivalent implementation strategies.

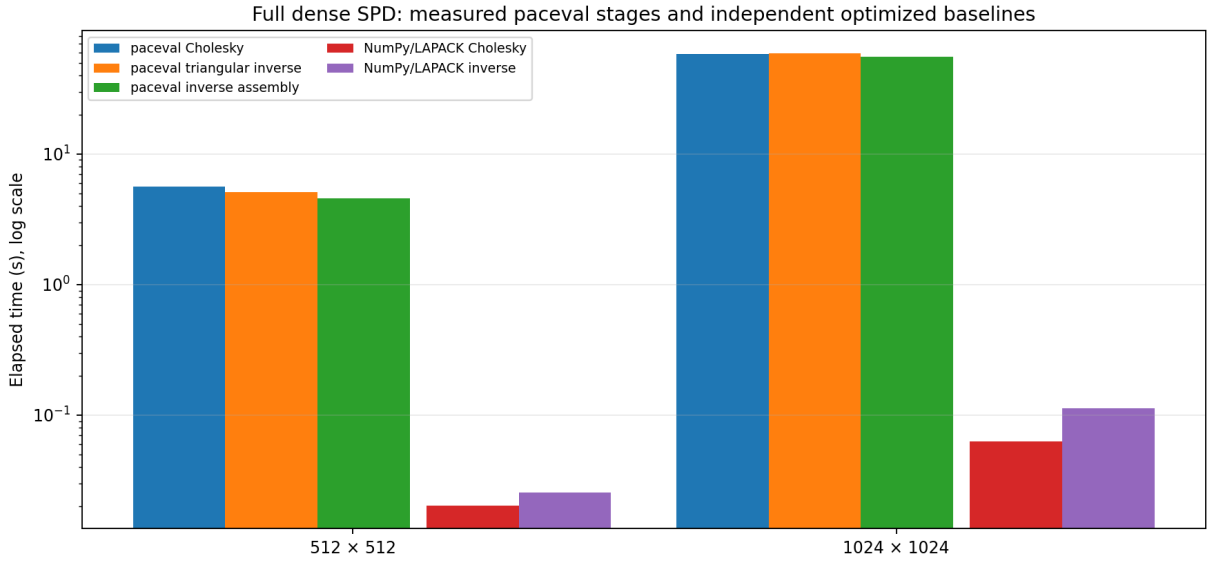


Figure 3: Measured paceval stages versus optimized numerical references

5.2 Accuracy and conditioning

Independent check	512 × 512	1024 × 1024
Smallest eigenvalue	1.02953	1.16624
Largest eigenvalue	3.38040	2.96162
Estimated condition number κ_2	3.28343	2.53946

Independent check	512×512	1024×1024
Cholesky difference versus independent native factor	0 in recorded path	0 in recorded path
Maximum factor difference vs generating L_0	2.66×10^{-15}	3.11×10^{-15}
Absolute inverse symmetry difference	0 (explicit mirroring)	0 (explicit mirroring)
paceval log determinant	317.103494576585	637.797955491835
Independent NumPy log determinant	317.103494576585	637.797955491837

The zero difference in the chosen floating-point Cholesky implementations should not be extrapolated to bitwise identity across compilers and processors. Similarly, zero inverse symmetry error follows from how the computed lower triangle was mirrored and is not an independent quality certificate. More informative tests are the full inverse residual, the all-element independent inverse comparison and the eigenvalue-based conditioning estimate.

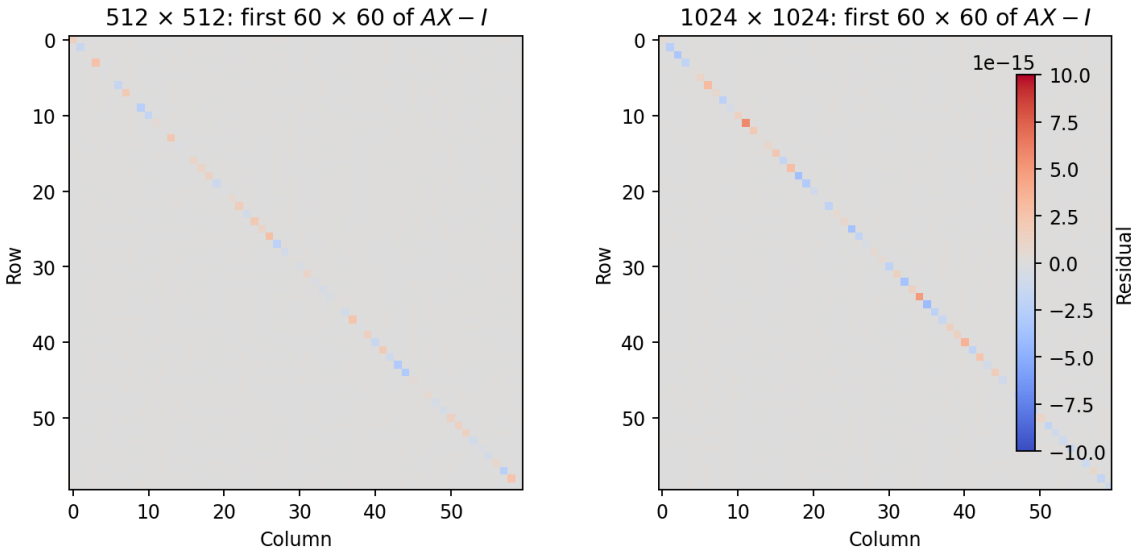


Figure 4: Small block of the complete residual matrices

Result: The new unstructured computation path is numerically valid for both tested, well-conditioned dense SPD matrices. It completes the previously missing full-size Cholesky and inverse experiments in this research environment.

5.3 Actual graph and batch execution counts

The new algorithm does not merely create a few graphs and then compute the matrix natively. Its core scalar arithmetic is performed by paceval for all pivots and all unique inverse entries. The stored logs contain the following execution counts:

Measured API activity	512×512	1024×1024
Cholesky-stage graph evaluations, including pivot/column operations	22,500,864	179,481,600
Subtract-multiply graph evaluations across Cholesky and triangular solves	44,739,072	357,913,600
Add-multiply graph evaluations for Gram inverse assembly	22,500,864	179,481,600

Measured API activity	512×512	1024×1024
All paceval batch calls across five kernels	18,721	529,361
Maximum configured batch capacity	4,096	1,024

The Cholesky-stage count includes some of the subtract-multiply evaluations also shown in the second row, so these rows **must not be added** to obtain an overall operation count. Instead, together with the individual metrics CSV, they document precisely which arithmetic was executed by the paceval engine. In this unblocked algorithm the Schur complement, forward-substitution updates and Gram inverse construction account for the dominant cubic workload. All outputs and per-element error codes were checked. Neither reported batch capacity is claimed to be optimal.

6. Interpretation for industrial use

The generic paceval implementation works but is **much slower** than specialized native and BLAS/LAPACK algorithms. This is not surprising: the current method sends each scalar update through a general expression engine, processes dense dependencies sequentially across pivots, and does not use the optimized blocked kernels of mature numerical libraries. The larger the matrix, the more decisive these disadvantages become.

In a production system, the preferred architecture is usually hybrid. Use a tuned library for bulk dense decompositions, matrix products and numerical solves, and assess paceval for application-specific, changing mathematical functions surrounding those numerical kernels. Examples include configurable post-Cholesky uncertainty rules, parametric error thresholds, combinations of several covariance-derived metrics, nonlinear condition scores and user-defined calibration constraints. In such workloads, the relevant hypothesis concerns **development and model-update effort, consistent graph reuse, integration convenience and end-to-end cost**, not a general acceleration of dense linear algebra.

Proposed real-world pilot

A sensible first industrial pilot would be a multi-sensor calibration or quality-control pipeline that ingests an actual covariance matrix, solves an SPD estimation problem with an optimized library, then applies a collection of frequently changed nonlinear mathematical checks through persistent paceval graphs. A fair experiment should compare not only total runtime and precision but also model revision time, number of code changes, validation effort and workload portability. The benchmark must clearly distinguish costs from computing the matrix primitive and costs from updating user-specific mathematics.

Research extensions with explicit stop criteria

1. **Numerically difficult matrices:** test controlled condition numbers, SPD matrices with clustered eigenvalues and failure cases with zero or negative pivots. Do not interpret these untested regimes as solved by this paper.
2. **Blocked kernel experimentation:** try matrix-matrix update graphs for short dense tiles, varied batch capacities and expression fusion; compare performance without assuming improvement.
3. **Multiple right-hand sides:** test generic SPD solves directly without materializing the inverse when the application only needs solutions; this is ordinarily preferable numerically and computationally.
4. **Industrial model-management value:** measure the engineering cost of modifying and verifying ten representative formula revisions. Reject the proposed integration if changes in formula maintenance do not offset runtime, memory and operational overhead.

7. Reproduction and audit trail

The new productive implementation is `src/dense_spd_paceval.cpp`. It uses the C++ standard library, system dynamic loading, and actual paceval computation objects. Core arithmetic graphs are `sqrt(a)`, `a/b`, `a-b*c`, `a+b*c`, and `ln(a)`. Input, output and error arrays are preallocated and reused for many `dGetComputationResultExt` calls. None of the productive Cholesky/inversion operations invokes NumPy or an external numerical solver.

From a Linux x86-64 machine with a separately supplied compatible `libpaceval.so`, run:

```
bash scripts/run_generic_spd.sh /absolute/path/to/libpaceval.so 512 4096
bash scripts/run_generic_spd.sh /absolute/path/to/libpaceval.so 1024 1024
```

The reproduction script writes regenerated results into `results/reproduced/` rather than overwriting the measured reference data. The external validation is implemented in `validation/verify_dense_numpy.py` using NumPy. It loads the *entire* original SPD matrix and the full paceval-computed inverse from IEEE-754 binary64 little-endian row-major `.f64` files. It independently calculates the full matrix inverse, Cholesky factorization, eigenvalues, a log determinant and $AA^{-1} - I$. Native C++ also computes a complete residual internally. The binary array format is documented and no binary paceval library is redistributed.

The package retains source and results for the earlier full dense products, structured covariance transformation, and low-rank SPD inversion. Each family remains separately named, with the previous restricted scope preserved. Existing data sets and diagrams are included to make the development from specialized to general dense SPD methods auditable.

8. Scientific status and final conclusions

This study establishes **actual numerical feasibility** of full 512 and 1024 generic dense SPD Cholesky decompositions and inverses through the real paceval C API. Every unique inverse entry was calculated using paceval, and complete matrices passed independent validation. A controlled illustration of paceval's model programmability is therefore available beyond the earlier diagonal-plus-low-rank Woodbury example.

It does **not** demonstrate improved performance, certify arbitrary conditioning regimes, show that the generic matrix solver is preferable to LAPACK, or establish a commercial benefit. The measured runtime gap is substantial. For industrial applications, the most attractive next experiment is the use of paceval as a **configurable, reusable mathematical execution layer alongside optimized matrix solvers**, with falsifiable metrics for changes in formula complexity, total processing time, maintainability, reproducibility and quality.

References and technical resources

- paceval Mathematical Engine, public API and research repository: https://paceval.com/; https://github.com/paceval/paceval.
- Netlib, *LAPACK* (including standard Cholesky and dense linear algebra routines): https://www.netlib.org/lapack/.
- Netlib, *BLAS*, standardized dense linear algebra primitives: https://www.netlib.org/blas/.
- NumPy, *Linear algebra* user reference: https://numpy.org/doc/stable/reference/routines.linalg.html.
- Eigen, open-source C++ matrix library: https://eigen.tuxfamily.org/.
- Prior study: `docs/Previous_Matrix_Study_EN.md`, archived in this research package with its original numerical test data.

All timings, matrices, figures and claims referring to actual tests in this report originate from the recorded experiments in the downloadable research package. No industrial deployment is claimed.